

TRABAJO PRÁCTICO 2

Seminario de Lenguajes opción .NET 1º Semestre - 2026

Importante

El presente trabajo práctico representa la **Fase 2 (y entrega final)** del Sistema de Gestión de Expedientes (SGE). Consiste en la evolución directa del núcleo de negocio desarrollado en el TP1, integrándolo con almacenamiento persistente real, un esquema robusto de seguridad y una interfaz de comunicación moderna basada en servicios web.

- **Carácter del trabajo:** Obligatorio y requisito excluyente para la aprobación de la materia.
- **Modalidad:** Grupal (manteniendo los mismos equipos del TP1).

Entrega

- **Fecha límite:** 28/06/2026 hasta las 23:59 hs.
- **Vía de entrega:** Formulario de Google (a publicarse oportunamente).
- **Formato:** Un único archivo comprimido .zip. El nombre del archivo zip debe contener los apellidos de los autores del trabajo, ejemplo: Apellido1_Apellido2_Trabajo2.zip. Este archivo debe contener:
 1. La solución completa de la aplicación en .NET. **Importante:** Previamente a la compresión del archivo, borrar todas las carpetas bin y obj de los cuatro proyectos para que la entrega sea más liviana.
 2. La base de datos SQLite inicializada (SGE.sqlite) con el administrador semilla y algunos usuarios de prueba precargados.
 3. Un documento explicativo (README.md o .pdf) detallando el orden de prueba de los endpoints desde **Scalar**, incluyendo las credenciales (correo y contraseña) del administrador y de los usuarios de prueba.

1. Estructura de la Solución (Evolución de Capas)

La solución seguirá estructurada en cuatro proyectos, modificando la capa de presentación:

- SGE.Dominio (Class Library - Sin dependencias externas)
- SGE.Aplicacion (Class Library - Depende de SGE.Dominio)
- SGE.Infraestructura (Class Library - Depende de SGE.Aplicacion y SGE.Dominio)
- SGE.WebApi (Web API de ASP.NET Core - **Reemplaza a SGE.Consola** y actúa como la *Composition Root*)

1.1 Modularización sugerida (Program.cs Limpio)

Para favorecer la mantenibilidad de la aplicación, **se sugiere** delegar el registro de dependencias y el mapeo de rutas mediante **Métodos de Extensión**. Su implementación es opcional, sin embargo, los animamos a intentarlo:

- **En SGE.Aplicacion:** Se recomienda crear una clase estática de extensiones para registrar Casos de Uso y Servicios de Aplicación (ej: `.AddAplicacion()`).
- **En SGE.Infraestructura:** Se recomienda crear una clase estática de extensiones para configurar la base de datos SQLite con EF Core, la Unidad de Trabajo y los repositorios reales (ej: `.AddInfraestructura()`).
- **En SGE.WebApi:** Se sugiere que los endpoints no se escriban todos directamente en el Program.cs. Pueden intentar agruparlos en clases estáticas por contexto de negocio (ej: `ExpedientesEndpoints.cs`, `UsuariosEndpoints.cs`, etc.) utilizando la interfaz `IEndpointRouteBuilder`.

2. Nuevos Requerimientos de Dominio (SGE.Dominio)

Conservaremos las entidades Expediente y Trámite. Expandiremos el dominio para dar soporte a la seguridad.

2.1 Entidad: Usuario

Definir la entidad de dominio para representar a los usuarios del sistema:

- **Id** (Guid)
- **Nombre** (string)
- **CorreoElectronico** (string)
- **ContrasenaHash** (string - Contraseña cifrada)
- **EsAdministrador** (bool - Flag que identifica al usuario con privilegios de gestión)
- **Lista de Permisos** (Colección interna de tipos enumerativos Permiso)

Invariantes y Reglas de Diseño:

- Todos los datos son obligatorios y deben estar validados.
- **Gestión Segura de Contraseñas:** La contraseña nunca debe guardarse en texto plano. Las contraseñas no se almacenarán directamente en la base de datos; en su lugar, se utilizará una función de hash para mayor seguridad. **Almacenamiento del hash:** El hash de la contraseña debe almacenarse en la base de datos, nunca la contraseña en sí. Para verificar la identidad del usuario al iniciar sesión, se vuelve a calcular el hash de la contraseña ingresada y se compara con el hash almacenado. Si los hashes coinciden, el usuario ha ingresado la contraseña correcta y se le permite acceder al sistema. *Aclaración adicional:* la contraseña original (en texto plano) solo viaja desde el cliente a la API durante el registro y el login; para el resto de las consultas u operaciones protegidas ya no es necesaria, ya que la identidad viaja de forma segura dentro del token JWT.
- El dominio debe proveer métodos públicos para gestionar la asignación de permisos de forma segura.

2.2 Enumerativo: Permiso

Conservaremos el enumerativo Permiso definido en el TP1 correspondiente a las acciones sobre el negocio de expedientes y trámites:

- ExpedienteAlta
- ExpedienteBaja
- ExpedienteModificacion
- TramiteAlta

- TramiteBaja
- TramiteModificacion

3. Capa de Aplicación y Casos de Uso (SGE.Aplicacion)

3.1 El Patrón Unit of Work (Unidad de Trabajo)

Definir la abstracción de la Unidad de Trabajo para asegurar la transaccionalidad:

```
public interface IUnidadDeTrabajo
{
    void Guardar(); // Confirma de forma atómica los cambios en la base de datos
}
```

Regla de Oro: Los repositorios **no** guardan cambios por sí mismos; solo marcan las entidades en memoria (Agregar, Modificar, Eliminar). El Caso de Uso mutativo es el único responsable de invocar a IUnidadDeTrabajo.Guardar() al finalizar su lógica con éxito.

3.2 Casos de Uso Obligatorios

Gestión de Expedientes y Trámites:

- **Alta, Modificación de Carátula, Cambio de Estado Manual y Baja en Cascada de Expedientes.**
- **Alta, Modificación y Baja de Trámites** (Orquestando la mutación automática del expediente mediante el servicio ActualizacionEstadoExpedienteService desarrollado en el TP1).
- **Consultas de lectura:**
 - **Listar todos los Expedientes.**
 - **Listar Trámites por Expediente** (Dado un ExpedientId).
 - **Obtener Expediente por Id (con detalle de Trámites):** Dado un Id de expediente, este caso de uso deberá retornar toda la información del expediente junto con la

colección completa de los trámites que posee asociados.

Gestión de Usuarios y Autenticación:

- **RegistrarUsuarioUseCase:** Registro abierto para cualquier persona (define correo, nombre y contraseña). **El caso de uso debe verificar primero que el correo electrónico ingresado no se encuentre ya registrado en la base de datos; de ser así, debe lanzar una excepción de negocio.** Por defecto, EsAdministrador es false y no cuenta con ningún permiso de mutación (solo lectura).
- **LoginUseCase:** Valida credenciales (**utilizando el correo electrónico como identificador único para buscar al usuario en la base de datos**) y retorna un token JWT que transporta la identidad básica del usuario (UserId).
- **ModificarMisDatosUseCase:** Cualquier usuario autenticado puede invocar este caso de uso para actualizar sus propios datos personales o cambiar su contraseña. El sistema debe validar estrictamente que el UserId extraído del token coincida con el usuario que se está intentando modificar (un usuario no puede editar los datos de otro).
- **Operaciones Exclusivas del Administrador:**
 - **ListarUsuariosUseCase:** Devuelve la lista completa de usuarios del sistema.
 - **EliminarUsuarioUseCase:** Da de baja a un usuario del sistema.
 - **ModificarPermisosUsuarioUseCase:** Permite asignar o remover permisos del enumerativo a un usuario específico.
 - *Regla de Control:* Estos tres casos de uso deben verificar en primera instancia si el usuario que ejecuta la acción posee el flag EsAdministrador == true. De lo contrario, se debe denegar la acción de inmediato lanzando una AutorizacionException.

3.3 Regla de Implicancia de Permisos

En el servicio de autorizaciones (IAutorizacionService), se debe mantener la regla: **El permiso ExpedienteBaja implica implícitamente contar con el permiso TramiteBaja.**

4. Capa de Infraestructura (SGE.Infraestructura)

4.1 Base de Datos con EF Core y SQLite

- Crear la clase SgeContext heredando de DbContext.
- Configurar el mapeo de las entidades hacia las tablas de SQLite. Utilizar el método **ComplexProperty** (dentro de OnModelCreating) para mapear los Value Objects.
- Inicializar la base de datos usando context.Database.EnsureCreated().

4.2 Datos Semilla (Seed)

En el momento de la creación de la base de datos, el sistema debe crear automáticamente un conjunto de usuarios iniciales:

1. **Un Administrador Semilla:**
 - **Correo Electrónico:** admin@sge.com
 - **Contraseña:** admin123 (Cifrada con hash)
 - **EsAdministrador:** true
2. **Al menos dos Usuarios de Prueba:** Crear un par de usuarios extra (por ejemplo, uno con permisos parciales y otro sin permisos) para que el evaluador pueda testear rápidamente el control de accesos sin tener que registrarlos a mano (documentarlos en el README).

4.3 Servicio de Autorización Definitivo

Reemplazar el AutorizacionProvisionalService desarrollado en la Fase 1 por su implementación real (por ejemplo, AutorizacionService). Este servicio utilizará un IUserRepository inyectado para buscar al usuario por su Id y verificar de forma efectiva si la colección interna de permisos de dicho usuario contiene el permiso que el Caso de Uso está solicitando para dejarlo operar. (Recordatorio: Al implementar la validación, tener en cuenta la regla de implicancia de permisos detallada en el punto 3.3 de este documento).

5. Capa de Presentación (SGE.WebApi)

- **Pipeline de Middlewares:** Configurar adecuadamente la tubería en Program.cs:
UseExceptionHandler() → UseAuthentication() → UseAuthorization().
- **Manejador de Excepciones Globales:** Interceptar las excepciones de negocio

(DominioException, AutorizacionException, EntidadNoEncontradaException) y retornar respuestas estandarizadas utilizando el formato industrial ProblemDetails con su código HTTP correspondiente (400, 403, 404).

- **Autenticación por JWT:** Configurar el middleware AddJwtBearer para validar los tokens emitidos en el login de forma estricta.
- **Documentación con Scalar:** Utilizar .WithTags() para organizar la interfaz gráfica y asegurar que las rutas protegidas requieran obligatoriamente el token JWT. El ID del usuario para las operaciones mutativas debe extraerse siempre del parámetro ClaimsPrincipal user (o mediante un contexto inyectado).

Apéndice de lectura - Notas sobre SQLite y sobre Hashes

Apartado sobre SQLite.

NOTA: Es conveniente establecer la propiedad *journal mode* de la base de datos sqlite en *DELETE*.

Se puede establecer por código de la siguiente manera:

```
if (context.Database.EnsureCreated())
{
    // Establecemos la propiedad journal_mode
    // de la base de datos SQLite en DELETE
    var connection = context.Database.GetDbConnection();
    connection.Open();
    using (var command = connection.CreateCommand())
    {
        command.CommandText = "PRAGMA journal_mode=DELETE;";
        command.ExecuteNonQuery();
    }
    . . .
}
```

Explicación de esta recomendación

La propiedad *journal_mode* se utiliza en una base de datos SQLite para especificar el modo de registro de transacciones que se utilizará. Éste determina cómo se guardan y administran los cambios realizados en la base de datos.

Al utilizar el modo *DELETE* en SQLite, los cambios realizados se reflejan inmediatamente en la base de datos principal. Esto significa que, después de confirmar una transacción, los datos modificados se guardan directamente en el archivo de la base de datos y están disponibles para su lectura inmediata.

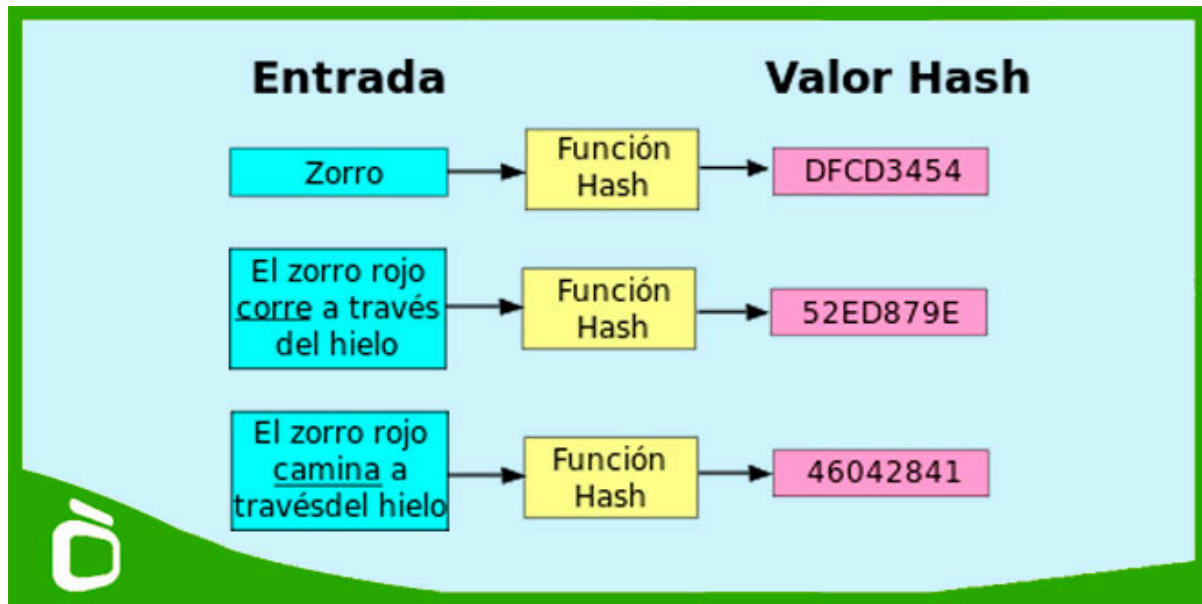
Por otro lado, en otros modos, que implican el uso de registros de transacciones, los cambios no se aplican directamente a la base de datos principal. En cambio, se escriben primero en el archivo de registro de transacciones. Luego, en segundo plano, se realizan las operaciones de fusionar y aplicar los cambios al archivo de la base de datos principal.

Como resultado, si se accede a la base de datos con otra herramienta mientras se utiliza un modo de registro de transacciones, es posible que no se vean los cambios reflejados inmediatamente. Es necesario esperar a que se realice la fusión y aplicación de los registros de transacciones antes de que los cambios sean visibles en la base de datos principal. La frecuencia con la que se realiza este proceso de fusión y aplicación puede variar y depende de factores como la configuración y la carga de trabajo del sistema.

Por lo tanto, el modo *DELETE* proporciona una escritura directa y visible en la base de datos, mientras que otros modos de registro pueden introducir una latencia en la propagación de los cambios debido a las operaciones de fusión y aplicación que deben llevarse a cabo.

Apartado sobre función de hash.

Una función hash es un procedimiento que transforma una información determinada (por ejemplo, un texto) en una secuencia alfanumérica “única” de longitud fija, denominada hash (resumen).



Un hash no es un cifrado porque su resultado no puede descifrarse para obtener el original. El proceso es irreversible.

Es útil para comprobar la integridad de la información. Si volvemos a calcular el hash y obtenemos el mismo valor significa, en la práctica, que la información no ha sido adulterada.

En este trabajo es necesario utilizar una función criptográfica de hash segura. Estas funciones deben cumplir algunos requisitos:

- **Eficiencia computacional:** El cálculo del hash debe ser computacionalmente eficiente.
- **Difusión o efecto avalancha:** un pequeño cambio en el input debe producir un cambio significativo en el output
- **Resistencia a la preimagen (Unidireccionalidad):** imposible "revertir" la función hash (encontrar el input a partir de un output determinado)
- **Resistencia a colisión:** La probabilidad de colisionar debe ser tan baja que requeriría millones de años de computaciones
- **Resistencia a la segunda preimagen:** Dado un hash debe ser inviable encontrar una input que produzca el mismo valor de hash

Son ejemplos de funciones de hash: MD5, SHA1, SHA-256, RIPEMD-160, etc.

SHA-256 produce un hash de 256 bits (32 bytes). Es una de las más usadas por su equilibrio entre seguridad y costo computacional de generación. El algoritmo SHA256 (Secure Hash Algorithm 256 bits) funciona dividiendo la entrada en bloques de 512 bits y sometiendo a cada bloque a una serie de rondas de procesamiento, que incluyen: rotaciones a nivel de bits, operaciones lógicas (AND, OR, XOR), adiciones modulares y mezclas de bits. En este link <https://sha256algorithm.com/> se puede observar al algoritmo SHA256 en funcionamiento.

¿Cómo utilizar SHA-256 en .NET?

Afortunadamente, no es necesario implementar SHA-256 desde cero en .NET, ya que la clase SHA256 del espacio de nombres System.Security.Cryptography proporciona la funcionalidad necesaria. Investigar sobre ella y utilizarla para obtener el hash de las contraseñas de los usuarios.